Question Bank — Week 4

Dynamic arrays and amortised analysis (Lecture 04) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice **1**

Part B — Short answer and essay **7**

Part C — Trace the growth **8**

Part D — Proofs **8**

Part E — Find and fix the bug **9**

Part F — Write the code **10**

> ***Answers are in a separate file:*** *`week04-answers.md`. Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify. Unless a question says otherwise, a dynamic array starts* ***empty with capacity 1****, and "doubling" means that an append which finds it full first doubles the capacity.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W4-M01 – W4-M40
B	Short answer and essay	W4-E1 – W4-E9
C	Trace the growth — show every step	W4-T1 – W4-T5
D	Proofs	W4-P1 – W4-P2
E	Find and fix the bug	W4-B1 – W4-B4
F	Write the code — checked by `pytest`	W4-C1 – W4-C3

Part A — Multiple choice

W4-M01 [what] For a `DynamicArray`, `len(a)` returns:

- **a)** the size — the number of elements stored
- **b)** the capacity of the `Array` underneath
- **c)** capacity minus size

- **d)** the number of resizes so far

W4-M02 [how] With doubling, what is the capacity after 5 appends?

- **a)** 5
- **b)** 6
- **c)** 8
- **d)** 16

W4-M03 [how] With doubling, how many resizes do 1,000 appends cause?

- **a)** 9
- **b)** 10
- **c)** 500
- **d)** 999

W4-M04 [how] If the capacity grows by **one** slot at every resize, the total number of copies for n appends is:

- **a)** $\Theta(n)$
- **b)** $\Theta(\log n)$
- **c)** $\Theta(n \log n)$
- **d)** $\Theta(n^2)$

W4-M05 [why] If the capacity grows by a constant **100** slots at every resize, the amortised cost of append is:

- **a)** $O(n)$ — the constant only divides the copying by 100
- **b)** $O(1)$ — 100 is large enough
- **c)** $O(\log n)$
- **d)** $O(100)$, which is $O(1)$

W4-M06 [what] With doubling, the amortised cost of append is:

- **a)** $O(\log n)$
- **b)** $O(1)$
- **c)** $O(n)$
- **d)** $O(n \log n)$

W4-M07 [why] With doubling, the **worst-case** cost of a single append is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n^2)$

W4-M08 [why] How does amortised cost differ from average-case cost?

- **a)** They are the same thing
- **b)** Amortised cost is always larger
- **c)** Amortised cost averages over random inputs
- **d)** Amortised cost averages over a sequence of operations and needs no assumption about the inputs

W4-M09 [how] With doubling, the total number of element **copies** for n appends is:

- **a)** less than 2n
- **b)** exactly $n \log_2 n$
- **c)** exactly $n^2/2$
- **d)** exactly n

W4-M10 [how] With doubling, which of these appends triggers a resize?

- **a)** the 16th
- **b)** the 17th
- **c)** the 18th
- **d)** the 32nd

W4-M11 [why] To shrink without thrashing, a dynamic array should halve its capacity when the size falls to:

- **a)** half the capacity
- **b)** capacity − 1
- **c)** one quarter of the capacity
- **d)** zero

W4-M12 [what] "Thrashing" in a dynamic array means:

- **a)** the array is full
- **b)** an append causes a resize
- **c)** the garbage collector runs
- **d)** alternating operations near one boundary make the array resize on almost every operation

W4-M13 [what] CPython's `list` grows its capacity by about:

- **a)** one eighth, plus a little
- **b)** one slot
- **c)** a factor of 10
- **d)** exactly double

W4-M14 [how] How can you see a Python list's capacity from Python?

- **a)** `len(values)`
- **b)** from `sys.getsizeof(values)`
- **c)** `values.capacity`
- **d)** it cannot be observed

W4-M15 [how] In a `DynamicArray` of size 5 and capacity 8, `a[-1]` reads `self._block` at index:

- **a)** 7
- **b)** −1
- **c)** 4
- **d)** it raises `IndexError`

W4-M16 [what] `insert_at(0, x)` on a dynamic array of size n costs:

- **a)** $O(1)$

- **b)** amortised $O(1)$
- **c)** $O(\log n)$
- **d)** $O(n)$

W4-M17 [what] `pop()` — removing the **last** element — costs:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n^2)$

W4-M18 [why] Why does growing by a factor give amortised $O(1)$ append?

- **a)** Because the array never needs to copy
- **b)** Because the copies form a geometric series, dominated by its last term, which is less than n
- **c)** Because Python optimises it
- **d)** Because resizes happen at random

W4-M19 [why] Growing by a factor of **1.5** instead of 2:

- **a)** makes append amortised $O(n)$
- **b)** makes append amortised $O(\log n)$
- **c)** keeps append amortised $O(1)$, with more copies but less wasted space
- **d)** removes the need for resizing

W4-M20 [how] With doubling and no shrinking, just after a resize, about how much of the capacity is unused?

- **a)** none
- **b)** a quarter
- **c)** three quarters
- **d)** about half

W4-M21 [what] Which structure in `dsa/` stores its items in the student's own `DynamicArray`?

- **a)** `Stack`
- **b)** `CircularQueue`
- **c)** `OpenAddressingHashMap`
- **d)** `BinarySearchTree`

W4-M22 [how] In the accounting ("bank") proof for doubling, how many coins must each append be charged?

- **a)** 1
- **b)** 3
- **c)** $\log_2 n$
- **d)** n

W4-M23 [how] With the course `DynamicArray` (capacity 1, doubling), what are `len(a)` and `a._capacity` after `a = DynamicArray(range(6))`?

- **a)** 6 and 6

- **b)** 8 and 8
- **c)** 6 and 8
- **d)** 6 and 12

W4-M24 [why] Why does `__getitem__` check `index < self._size` rather than `index < self._capacity`?

- **a)** The slots beyond `size` hold nothing the user stored, so reading one must raise `IndexError`, as it does for a Python list
- **b)** The slots beyond `size` do not exist in the `Array` yet
- **c)** Comparing with `size` is faster than comparing with `capacity`
- **d)** It is only needed so that negative indices work

W4-M25 [how] What does this print?

```
a = DynamicArray()
for i in range(9):
    a.append(i)
print(a.resize_count, a._capacity)
```

- **a)** `3 8`
- **b)** `9 16`
- **c)** `4 9`
- **d)** `4 16`

W4-M26 [what] When the course `DynamicArray` resizes, it:

- **a)** makes the existing `Array` longer, in place
- **b)** allocates a new, bigger `Array`, copies every element into it, and switches `_block` to it
- **c)** copies only the new element into a second `Array`
- **d)** links a second `Array` after the first one

W4-M27 [why] Why can `_resize` not simply make the existing `Array` longer?

- **a)** Python forbids changing an object after it is created
- **b)** A longer `Array` would make indexing $O(n)$
- **c)** An `Array` is one fixed-size block of memory; the memory just after it may already be in use, so a bigger block has to be allocated elsewhere
- **d)** The elements must be copied so that they stay sorted

W4-M28 [how] A `DynamicArray` holds 8 elements and has capacity 8. Counting the elements copied by a resize **and** the elements shifted, how many existing elements move during `insert_at(0, "x")`?

- **a)** 8
- **b)** 9
- **c)** 1
- **d)** 16

W4-M29 [how] `a = DynamicArray([10, 20, 30, 40, 50])`. What does `a.pop(2)` return, and how many elements move?

- **a)** 30; two elements shift one place left
- **b)** 30; no element moves
- **c)** 20; two elements shift one place left
- **d)** 30; three elements shift one place left

W4-M30 [how] After 9 appends and then 8 calls of `pop()`, what is the capacity of the course `DynamicArray`?

- **a)** 1 — popping gives the memory back
- **b)** 16 — the course pop never shrinks the array
- **c)** 2 — it halves whenever it is a quarter full
- **d)** 4 — it halves whenever it is half full

W4-M31 [how] During 1,000,000 appends to an array that doubles, the single most expensive append copies how many elements?

- **a)** 1,000,000
- **b)** 999,999
- **c)** 524,288
- **d)** 20

W4-M32 [how] With `DynamicArray(growth=3)`, the total number of elements copied by n appends is always less than:

- **a)** n / 3
- **b)** 3n
- **c)** $n \log_3 n$
- **d)** 1.5n

W4-M33 [what] Which of these statements is an **amortised** bound rather than a worst-case bound for a single operation?

- **a)** Any n appends to a doubling array cost fewer than 3n units of work in total
- **b)** One append can cost $O(n)$
- **c)** Every `a[i]` costs $O(1)$
- **d)** `insert_at(0, x)` costs $O(n)$

W4-M34 [what] `DynamicArray(growth=1.5)` fails at its first resize. Why?

- **a)** 1.5 is not greater than 1
- **b)** `1 * 1.5` is a float, and an `Array` needs a whole number of slots
- **c)** append would become amortised $O(n)$
- **d)** Python cannot multiply an `int` by a `float`

W4-M35 [why] What does `DynamicArray([1, 2], growth=1)` do?

- **a)** It works; the capacity grows by one slot at each resize
- **b)** It loops for ever
- **c)** It raises `IndexError` on the second append: the "resize" makes a new block of 1 × 1 = 1 slot, so there is still no room
- **d)** It raises `ZeroDivisionError`

W4-M36 [why] An array that grows by a constant **+10** slots copies 798,400 elements for 4,000 appends. For 8,000 appends it copies about:

- **a)** the same number — the step is the same
- **b)** twice as many
- **c)** 13 more
- **d)** four times as many

W4-M37 [what] A `DynamicArray` holds 5 elements in capacity 8. What does `a[5] = "x"` do?

- **a)** writes into slot 5, and the size becomes 6
- **b)** writes into slot 5, and the size stays 5
- **c)** raises `IndexError`
- **d)** appends `"x"` at the end

W4-M38 [why] In the accounting argument (3 coins per append), the doubling from capacity 8 to 16 copies 8 elements. Which coins pay for it?

- **a)** The 2 spare coins saved by each of the 4 appends made since the previous resize
- **b)** One coin from each of the 8 appends made so far
- **c)** All 3 coins of the append that triggers the resize
- **d)** Coins borrowed from the appends still to come

W4-M39 [why] A student's `_resize` copies into the new block and sets `self._block`, but forgets `self._capacity = capacity`. Starting empty with capacity 1, what happens?

- **a)** Nothing goes wrong: `_capacity` is only for display
- **b)** The second append raises `IndexError`
- **c)** Every append resizes from then on: slow, but correct
- **d)** The third append raises `IndexError`: the size never equals the stale capacity again, so the full block is never grown

W4-M40 [what] Which methods of the course `DynamicArray` can trigger a resize?

- **a)** only append
- **b)** append and `insert_at`
- **c)** append and pop
- **d)** append, `insert_at` and `__setitem__`

Part B — Short answer and essay

W4-E1 [why] *(5 marks)* Define amortised cost. Prove that n appends to a dynamic array that doubles cost $O(n)$ in total, and conclude the amortised cost of one append.

W4-E2 [why] *(4 marks)* Explain why growing the capacity by a constant number of slots gives $\Theta(n^2)$ total cost for n appends, however large the constant, while growing by a constant factor gives $\Theta(n)$.

W4-E3 [why] *(3 marks)* `append` is "worst-case $O(n)$ and amortised $O(1)$". Explain both statements, why both are true, and when the worst case still matters.

W4-E4 [why] *(4 marks)* Explain thrashing with a concrete sequence of operations, and the shrinking rule that prevents it.

W4-E5 [why] *(3 marks)* Growth factor is a time–space trade-off. Explain the trade-off, and why CPython's `list` uses a small factor (about 1.125).

W4-E6 [how] *(4 marks)* A `DynamicArray` holds `a b c d` in capacity 4. Describe every step of `insert_at(1, "x")`, give the final contents, size and capacity, and count the elements copied and shifted. Why must `insert_at` check for a full array **before** it shifts?

W4-E7 [why] *(4 marks)* `Stack`, `SlowQueue`, `MinHeap` and `Graph` are all built on your `DynamicArray`. For the stack and the slow queue, explain how the cost of each operation follows from the `DynamicArray` table. Why does a bug in `DynamicArray` show up in the tests of other weeks?

W4-E8 [why] *(3 marks)* Explain the accounting ("bank") argument for a doubling array: how many coins each append is charged, what each coin pays for, and why the bank never goes negative.

W4-E9 [why] *(4 marks)* A game appends one record to a dynamic array every frame and must never take longer than one frame. Explain why a doubling array is a risk even though append is amortised $O(1)$, and describe two ways to remove the risk.

Part C — Trace the growth

Doubling, starting empty with capacity 1, unless stated otherwise. Count one unit of work per element **written** by append and one per element **copied** by a resize.

W4-T1 [how] After 10 appends: what is the capacity, how many resizes have happened, and how many elements have been copied in total?

W4-T2 [how] For 20 appends, list which appends trigger a resize, and compute the total work (writes + copies) and the average work per append.

W4-T3 [how] Now the capacity grows by **+3** at each resize (1, 4, 7, 10, …). For 12 appends: which appends trigger a resize, what is the final capacity, and how many elements are copied?

W4-T4 [how] A shrinking dynamic array doubles when full and, **after** a pop, halves when size ≤ capacity // 4 (and capacity > 1). Starting empty, perform `append(1)` … `append(5)`, then `pop()` four times. Give the size and capacity after every operation.

W4-T5 [how] Among the first 40 appends with doubling, which ones are "expensive" (trigger a resize)? What pattern do their numbers follow?

Part D — Proofs

W4-P1 [why] *(4 marks)* A dynamic array grows by a factor of **4** (capacity 1, 4, 16, 64, …). Prove that n appends cost $O(n)$ in total.

W4-P2 [why] *(4 marks)* A dynamic array grows by a constant c slots. Prove that n appends copy at least $\frac{n^2}{2c} - n$ elements, and hence that the total cost is $\Omega(n^2)$ for any fixed c.

Part E — Find and fix the bug

Each method belongs to a `DynamicArray` with `_block` (an `Array`), `_size` and `_capacity`.

W4-B1 [how]

```
def append(self, value):
    if self._size > self._capacity:
        self._resize(self._capacity * 2)
    self._block[self._size] = value
    self._size += 1
```

W4-B2 [how]

```
def __getitem__(self, index):
    if index < 0:
        index += self._size
    if not 0 <= index < self._capacity:
        raise IndexError(index)
    return self._block[index]
```

W4-B3 [why] Passes every test except `test_doubling_keeps_resizes_logarithmic`.

```
def append(self, value):
    if self._size == self._capacity:
        self._resize(self._capacity + self.growth)
    self._block[self._size] = value
    self._size += 1
```

W4-B4 [why] Passes every test, yet holds on to memory it should not.

```
def pop(self, index=-1):
    if self._size == 0:
        raise IndexError("pop from empty")
    if index < 0:
        index += self._size
    value = self._block[index]
    for i in range(index, self._size - 1):
        self._block[i] = self._block[i + 1]
    self._size -= 1
    return value
```

Part F — Write the code

In `practice/week04.py`; check with `pytest tests/test_practice_week04.py -v`.

W4-C1 [how] `capacity_after(n, factor=2)` — the capacity after n appends.

W4-C2 [how] `copies_for(n, grow)` — how many elements n appends copy, when `grow(capacity)` gives the new capacity. Use it to check the numbers in Lecture 04.

W4-C3 [why] `ShrinkingArray` — a dynamic array on an `Array` that doubles when full and halves when a pop leaves it one quarter full. Why does `test_shrinking_array_does_not_thrash` pass with the quarter rule and fail with a half rule?